Lecture 33 - Software Evolution and Maintenance

Thursday, 21 May 2026 8:30 am

Two different approaches for Software Processes

- Both have tradeoffs (pros and cons) and depend on the context of the project
- Traditional
  - Closer to answering, **"What to do"**
  - Very common in government standards
  - Your goal is to deliver something but you are unsure whether it gives value to the stakeholders
  - Advantages
    - very prescriptive definition
    - Tells you who has to do what, with which resources
    - Can be very useful especially if you are just entering a team
    - Has order to tasks
  - Disadvantages
    - Its rigid structure makes it more difficult to adapt and be flexible
    - Difficult to change and innovate

- Agile
  - answers more for, **"How to do it"**
  - Your goal is to produce something by the end of the day and gives value to the stakeholders
  - Advantages
    - The team can be transparent and flexible to change
    - Most popular: Scrum
  - Disadvantages
    - Dependent on discipline of the people
    - The team must be transparent and flexible to change

SCRUM

- Has 3 Roles
  - Product owner
    - Expert in business context and tells the team what to do in the sense of what they need (the requirements
  - Developers
    - Ideally, you have more than one team
  - Scrum Master
    - Manages Scrum framework and maintaining that the scrum rules are followed
      - Follows up with the team that all the Scrum rules are being executed
    - Coaching the developers team in scrum

- Sprint Review
  - Meeting with product owner and stakeholder to show the increment (the new thing you added)

- Sprint Retrospective
  - A meeting with the developers and scrum master where the developers identify….
    - What went well
    - What could be improved
    - What they could keep doing
    - What they could try for the next iteration
  - Strongly related to the Inspection and Adaptation goals of SCRUM
  - Inspect and adapt for the next iteration
  - Sprints are short, the adaptation and inspection are frequent and ideally, we can optimise our process
    - In theory…
    - It's not that easy in real life, there are a lot of different techniques in retrospective

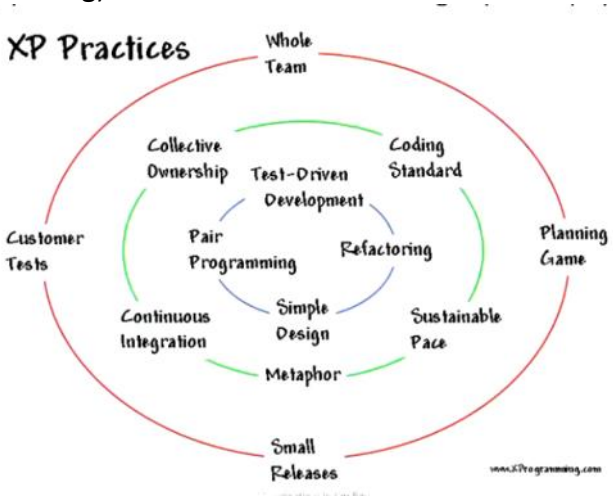
- You can also pick elements/practices from SCRUM
  - If the entire scrum process does not apply to your project to apply to your team's way of working for example…

Agile Approach

- User stories as requirements

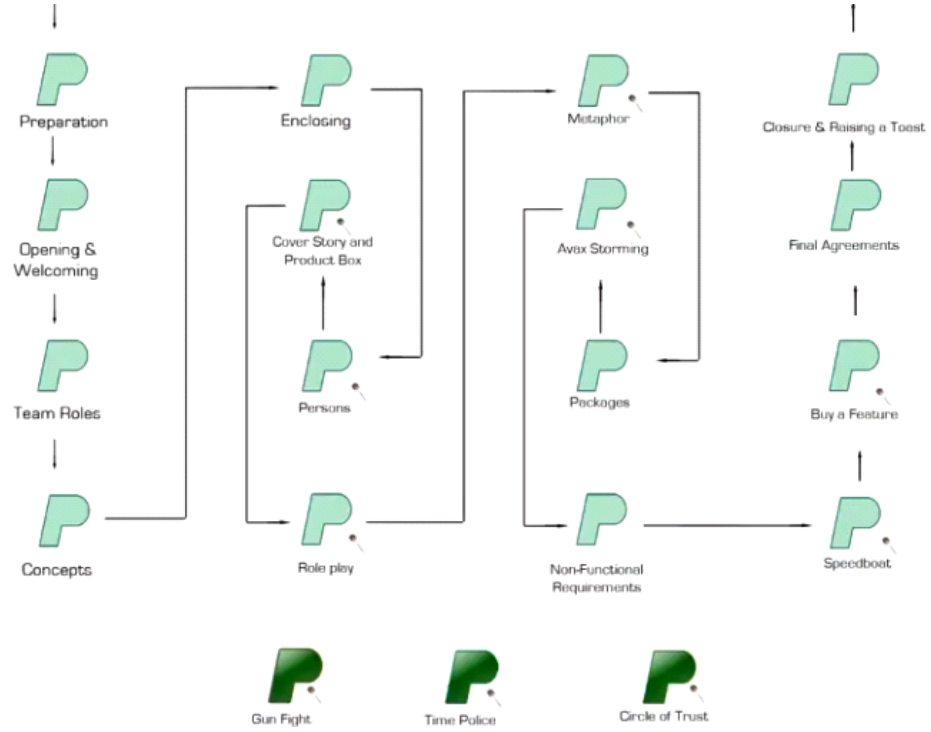
  - Using elements of SCRUM
    - Giving the team the power to determine or estimate how long a task will take with the size of the task
    - They will agree on the complexity of the task
    - Split the task into smaller parts to be able to determine a more accurate estimation for how many hours a task could take
    - User stories are taken as subset and placed into the **Product Backlog** and goes into the process of SCRUM

- XP (Xtreme Programming)



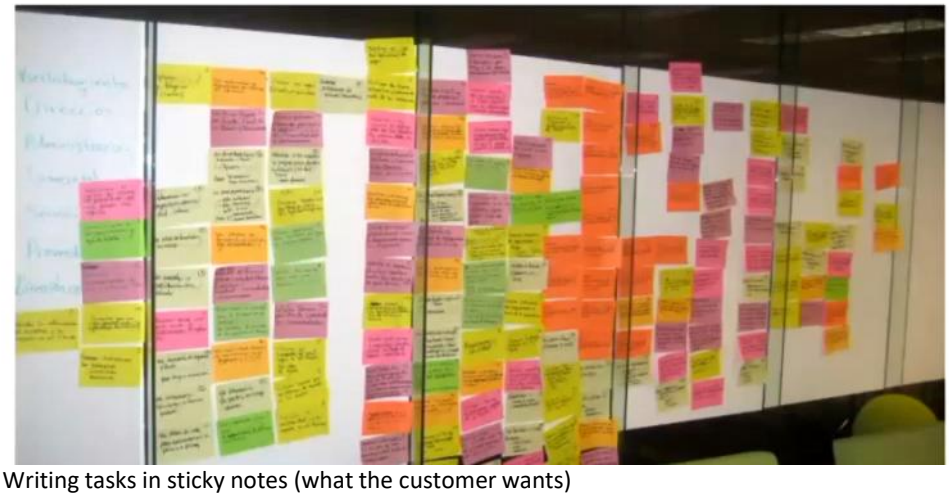
  - Gives practices that will make the development more efficient
  - Continuous integration
    - a good practice that doesn't just belong to SCRUM
  - Pair Programming
    - Sitting together with someone else and share what you are implementing (driver and passenger technique)
    - Usually you pair a senior and a junior programming
    - Good for learning
  - Test-Driven Development
    - Start by designing the test before implementing
    - Ensures that your program is tested and follows what you want to happen`

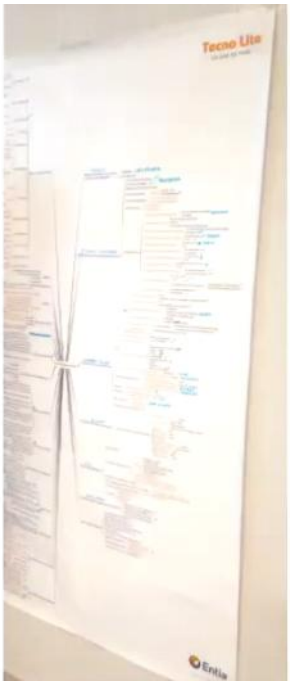
- Waterfall Approach / Games



- The lecturer mentions a company that has clients play a 'game' that visualises how their product would look like. They ask them to design a box where it would look similar to their product.
- Helped align the vision of the team with the clients and team which direction to go
- It took them around 12 hours for that workshop

- Sticky notes / Buy a feature





- Writing tasks in sticky notes (what the customer wants)
- Identify what features should be implemented (identify the requirements)
  - Discuss with the stakeholders
  - Requires having all the relevant stakeholders to this workshop and create post it notes together
  - Colours are based on what the stakeholders roles are (priorities)
- They are then grouped together
  - This example grouped the post it notes into **prices** which is the cost that would take to create that feature
  - They had an agreed fixed budget and makes the group discuss and decide which will be the key features (what they believe are the most important)
- This workshop is charged/paid hours

- *Potential Exam Questions*
  - What are the roles in SCRUM?
  - What is the purpose of a Kanban board?

Maintentance

- Software may require changes or adjustments that should be considered
- Some project don't consider maintenance, it requires good design and test practices
- Who does Maintenance?
  - Development team
  - The problem is most teams don't want to maintain their own system and work on it only during their role for the lifetime of the product

Learning Outcomes

- **Understand** and **remember** the concept of software evolution
- **Understand** and **remember** the concept of software maintenance
- **Know** the different types of software maintenance
- **Remember** some best coding practices and good design principles

Software Evolution

- Being a software engineer…
  - Imagine if you have to change source code written by others or even by yourself
  - Sometimes you can quickly identify the lines of code to change
  - The change is nicely isolated, and tests confirm that it works as intended
  - But at other times, a change creates more problems than it solves

- Evolution
  - Implies **substantial changes** to the application that **lead to a new release or a major version**
    - A change in the architecture to support adding a new functionality

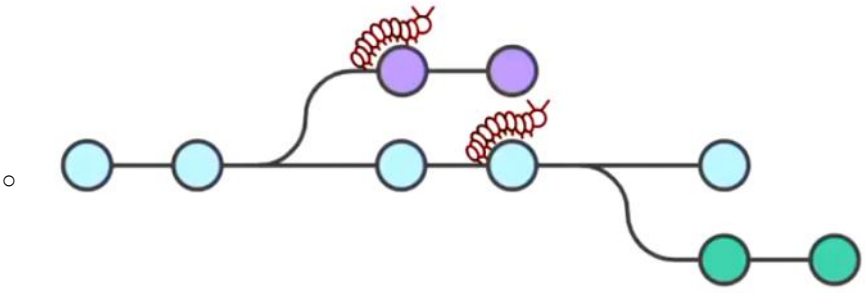
- Maintenance
  - Is about **minor changes** and may not result in a new release or version
    - Such as changing the data type
  - A change to improve performance of the application

- The ISO/IEC 12207 describes maintenance as:
  - ISO/IEC 1207 is the standard for software maintenance
  - The modification to code and associated documentation due to a problem or the need for improvement
    - *Must be meaningful documentation*
    - *Good design decisions*
      - why did you decide to use something?
      - Is it still meaningful to the current project? If not, then you need to document and make the changes again.
      - Failing to update documentation means you lose important knowledge
  - *The objective is to modify existing software products while preserving their integrity*

- Software maintenance can be classified into four types:
  - Corrective
    - **bugs** are discovered and need to be fixed
    - Something is not working as expected and needs to be fixed
    - Modification is minor
      - Such as typos and wrong data types
  - Adaptive
    - **changes in the context in which it operates**
    - Example: OS or technology upgrades
    - Example: you want to change to cheaper or open source technology
    - Nothing is wrong but you may need to upgrade how your application because the context is changing
  - Perfective
    - users of the system and/or other stakeholders have new or changed requirements
    - Example: biometrics or 2FA
    - Cybersecurity
  - Preventive

        - ways are identified to **increase quality or prevent future bugs** from occurring
        - Example: optimizing a routine because user traffic will increase
        - Example: adding a load balancer that wasn't needed previously
        - Increasing the quality of the product
            - the performance and availability will be better

Software Maintenance

- **Managing bug fixes across different versions** can become complicated very quickly
    - The more branches require you to identify where to fix bugs and manage fixes across different parts of the application
    - 

- Maintainability is only one of the eight characteristics of software product quality
    - Maintainability is **the degree of effectiveness and efficiency** with which a product or system can be modified by the intended maintainers
    - If it's hard to modify then it's hard to maintain (BAD!)

- Maintainability has five quality sub-characteristics:
    - *Modularity*
        - Put different features in groups
        - Makes them easily replaceable
    - *Reusability*
        - Allows you to only need to change something only once
    - *Analise-ability*
        - Degree in which you can determine how much effort something needs to change (readable and clarity)
    - *Testability*
        - Be able to demonstrate that it works properly and intentionally
    - *Modifiability*

- *You will usually have a question at the end of the exam about Software Maintenance*

Moral Homework

- Research the main characteristics of a software product in ISO/IEC 25010
    - According to the lecturer, the ones discussed are outdated because they recently updated it to 9 qualities